8051 – Intrebari de teorie posibile

- **Arhitectura**

only 1 On chip oscillator (external crystal)

6 interrupt sources (2 external , 3 internal, Reset)

64K external code (program) memory(only read)PSEN

64K external data memory(can be read and write) by

RD,WR

Code memory is selectable by EA (internal or external)

We may have External memory as data and code

<u>Porturi de interes:</u>

Vcc（pin 40）：

Vcc provides supply voltage to the chip.

The voltage source is +5V.

GND（pin 20）：ground

XTAL1 and XTAL2（pins 19,18）：

These 2 pins provide external clock.

Way 1：using a quartz crystal oscillator

Way 2：using a TTL oscillator

<u>Alte porturi de interes:</u>

PSEN (out): Program Store Enable, the read signal for external program memory (active low).

ALE (out): Address Latch Enable, to latch address outputs at Port0 and Port2

EA (in): External Access Enable, active low to access external program memory locations 0 to 4K

RXD,TXD: UART pins for serial I/O on Port 3

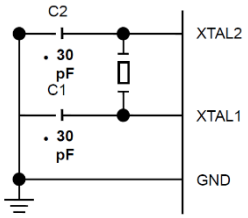
XTAL1 & XTAL2: Crystal inputs for internal oscillator.

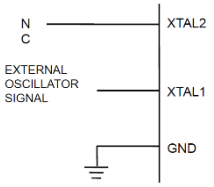
<u>Porturi I/O:</u>

<u>XTAL:</u>

Folosind un oscilator de cuartz

Frecventa poate fi masurata pe XTAL2





Folosind un oscilator TTL

XTAL2 este liber.

<u>RESET:</u>

-RST（pin 9）：reset

Pin de intrare si activ 1 （normal 0）.

Lungimea pulsului trebuie sa fie de cel putin 2 cicluri de tact.

-power-on reset

Dupa pulsul de RSET, 8051 se va reseta si toate valorile din memorie vor fi pierdute. Se vor reseta valorile registrilor (inclusiv cei speciali)

ALTE PORTURI DE I/O SE GASESC IN CURSURILE CONCATENATE (crtl+f pe Alte Porturi de I/O).

<u>Informatii despre Porturi de I/O in general:</u>

- Fiecare dintre aceste porturi
  - Este conectat intern la BUS-ul CPU
  - Un D latch stocheaza valoarea pin-ului. Daca write_to_latch = 1 atunci data se scrie prin latch.
  - 2 buffere Tri-state :
    - TB1: controlat de “Read pin”
      - Read pin＝1 : citeste data de pe pin

      TB2: controlat de “Read latch”

      Read latch＝1 : citeste valoarea din pin

    O poarta cu tranzistor (M1)

    Gate=0: open

    Gate=1: close

- **Tipuri de conectare a memoriilor externe**

Prin memorie externa se intelege orice tip de memorie ce nu este incapsulata in microcontroller. Utilizarea memoriilor externe implica mai multe operatii decat in cazul memoriei interne, motiv pentru care operatiunile cu aceasta se desfasoara mai incet.

De exemplu, pentru a incrementa valoarea de la o adresa se foloseste doar 1 instructiune in cazul memoriei interne pe intervalul unui tact de ceas. In cazul memoriei externe, este nevoie de 4 instructiuni ce se executa in 7 tacturi de ceas motiv pentru care utilizarea memoriei externe poate duce la cresteri de pana la 7 ori a timpului de executie.

In timp ce memoria interna a MCU-ului are doar 128 bytes memoria externa poate fi adresara pana la dimensiunea de 64k.

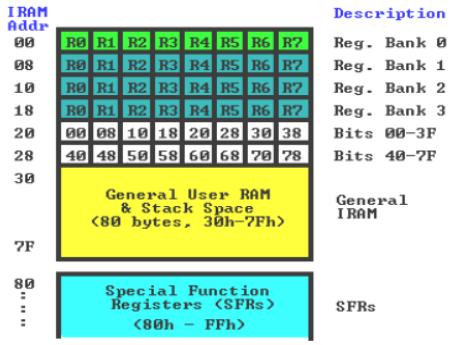
Memoria de pe chip

Unitatea MCU 8051 are o memorie interna volatila de 128 bytes, ei fiind accesibili sistemului cu viteza maxima. Acestia sunt impartiti in urmatoarele grupe

1) Primi 8 octeti (00h-07h) reprezinta primul bank de registrii (bankul 0). Prin utilizarea registrilor speciali (SFR) se poate selecta cate un bank din seria 1,2 sau 3 (08h-1Fh).

2) Memoria pe biti (20h-2Fh)

3) Zona de stocare a variabilelor (sau stiva) (30h-7Fh) este o zona in care variabilele si stiva o impart.



Modurile de accesare a memoriei

Prin modalitatea de accesare a memoriei se intelege modul cum referim o anumita locatie din memorie. Tipurile de accesare a memoriei sunt:

Immediate Addressing	MOV A,#20h
Direct Addressing	MOV A,30h
Indirect Addressing	MOV A,@R0
External Direct	MOVX A,@DPTR
Code Indirect	MOVC A,@A+DPTR

Tipuri de adresare (conectare):

- Adresarea Imediata:

Este denumita asa deoarece valoarea ce urmeaza a fi scrisa in memorie, urmeaza imediat instructiunii, ceea ce se traduce prin faptul ca instructiunea in sine dicteaza ce anume se va scrie in memorie.

MOV A,#20h

Scrie constanta 20h in registrul acumulator. Acest tip de instructiune are un timp scazut de executie tocmai datorita faptului ca valoarea ce urmeaza a fi scrisa este fixata la momentul compilarii.

Totusi acest mod de adresare nu confera flexibilitatea necesara implementarilor cu un grad ridicat de complexitate.

- Adresarea Directa:

Este denumita asa deoarece valoarea ce urmeaza a fi scrisa in memorie se afla in memoria interna a MCU-ului sau poate fi obtinuta prin accesarea directa a acesteia:

MOV A,30h

Scrie valoarea de la adresa 30h in registrul acumulator. Acest tip de instructiune are un timp scazut de executie deoarece refera doar memoria interna a MCU-ului.

In cazul implementarilor ulterioare (8052 – 256 octeti memorie) prezinta limitari la accesarea memoriei peste 128 de octeti

- Adresarea Indirecta:

Adresarea indirecta este cea mai flexibila modalitate de adresare a memoriei MCU si poate fi utilizata pentru accesarea memoriei de peste 128 de octeti (8052).

MOV A,@R0

Aceasta instructiune rezulta in evaluarea valorii din R0 si asignarea acumulatorului cu valoarea gasita in memorie la adresa indicata de R0.

De ex. Daca R0 indica valoarea 40h iar in memorie la adresa 40h se afla inregistrata valoarea 67h, la executia instructiunii de mai sus, acumulatorul va prelua valoarea 67h (valoarea de la adresa stocata in R0).

Prin adresare directa poate fi accesata memoria interna dar niciodata un registru special.

- **Cate nivele de întreruperi in modul de trigger, care si cate sunt prioritatile, si de ce sunt necesare**

- 8051 suporta 5 intreruperi, 2 interne(Timer), 2 externe si una seriala

1) External 0

2) Timer 0

3) External 1

4) Timer 1

5) Serial Interrupt

Criterii dupa care se stabileste prioritatatea intreruperilor:

- O intrerupere de nivel innalt nu poate fi intrerupta nici macar de o alta intrerupere de acelasi nivel

- O intrerupere de nivel innalt poate intrerupe executia intreruperilor de nivel mai mic

- O intrerupere de nivel scazut poate aparea doar daca nu se executa o intrerupere de nivel inalt

- Daca doua intreruperi au loc simultan se executa cea mai importanta prima data. In caz de egalitate, executia are loc in ordinea verificarii:

1) External 0

2) Timer 0

3) External 1

4) Timer 1

5) Serial Interrupt

De ce sunt necesare prioritatile?

Ca sa se stabileasca o ordine.

- **Set de instructiuni**

<u>Intstructiuni de transfer:</u>

Exemplu: MOV dest, source (dest<-source)

Instructiuni Stack

PUSH byte (increment stack pointer, move byte on stack)

POP byte (move from stack to byte, decrement stack pointer)

Instructiuni Exchange

XCH a, byte (exchange accumulator and byte)

XCHD a, byte (exchange low nibbles of , accumulator and byte)

two way data transfer

XCH a, 30h ; a <-> M[30]

Transferul se face individual pe fiecare bit.

Bitii de Ram de la adresele 20-2F sunt adresabili

<u>Modalitati de adresare:</u>

Adresarea Imediata

mov B, #11 (put 11 decimal in b register, B = 00001011)

Adresarea registrilor (sursa sau destinatia este una din registrul CPU)

MOV R0,A

MOV A,R7

Adresare Directa (specificati datele dupa adresa de 8 biti. Folosita de obicei pentru adresele 30h-7Fh din Ram)

Mov a, 70h (copy contents of RAM at 70h to a)

Mov R0,40h (copy contents of RAM at 70h to a)

Adresare indirecta(adresa sursei sau destinatiei e specificate in registrii)

Foloseste registrii R0 sau R1 pentru adresele de 8 biti:

mov psw, #0 (use register bank 0)

mov r0, #0x3C

mov @r0, #3 (memory at 3C gets #3, M[3C] <- 3)

Foloseste registrii DPTR pentru adresele de 16 biti:

mov dptr, #0x9000 (dptr <- 9000h)

movx a, @dptr (a <- M[9000])

Instructiuni aritmetice:

- ADD A, byte – aduna A la byte, pune rezultatul in A
- ADDC A, byte – aduna cu carry
- SUBB A, byte – scadere cu borrow
- INC A, byte – incrementeaza A
- INC byte – incrementeaza byteul din memorie
- INC DPTR – incrementeaza pointerul de date
- DEC A – decrementeaza acumulatorul (registrul acumulator?)
- DEC byte – decrementeaza acumulatorul
- MUL AB – inmulteste acumulatorul cu registrul B
- DIV AB – imparte acumulatorul cu B
- DA A – decimal adjust

Logice:

- ANL – AND
- ORL – OR
- XRL – XOR
- ? - NOT
- CLR – clear
- RL – rotate left
- RLC – rotate left through Carry
- RR – rotate right
- RRC – rotate right through Carry
- SWAP – swap accumulator nibbles

Instructiuni de salt:

- Unconditional jumps (“go to”)

- Conditional jumps

- Call and return

Instructiunile de salt neconditionate:

- SJMP <rel addr> ; Short jump, relative address is 8-bit 2’s complement number, so jump can be up to 127 locations forward, or 128 locations back.

- LJMP <address 16> ; Long jump

- AJMP <address 11> ; Absolute jump to anywhere within 2K block of program memory - JMP @A + DPTR ; Long indexed jump

Instructiunile de salt conditionate:

JZ <rel addr> Jump if a = 0

JNZ <rel addr> Jump if a != 0

JC <rel addr> Jump if C = 1

JNC <rel addr> Jump if C != 1

JB <bit>, <rel addr> Jump if bit = 1

JNB <bit>,<rel addr> Jump if bit != 1

JBC <bir>, <rel addr> Jump if bit =1, &clear bit

CJNE A, direct, <rel addr> Compare A and memory, jump if not equal

Tabel de look up

- **Explicati modalitatea de functionare a unei porti (AND OR XOR) folosind tabelul de look up**
- **Schema + tabel pentru functie (asta e raspunsul efectiv o schema pentru functie si tabelul de valori)**